Estruturas de Dados Básicas II

Prof. Dr. Matheus da Silva Menezes



UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Continuando...

Estruturas de Dados Básicas II

Notação Assintótica - Big O

Análise de Algoritmos

Em 1965 Jack Edmonds introduz a ideia de Complexidade Assintótica.

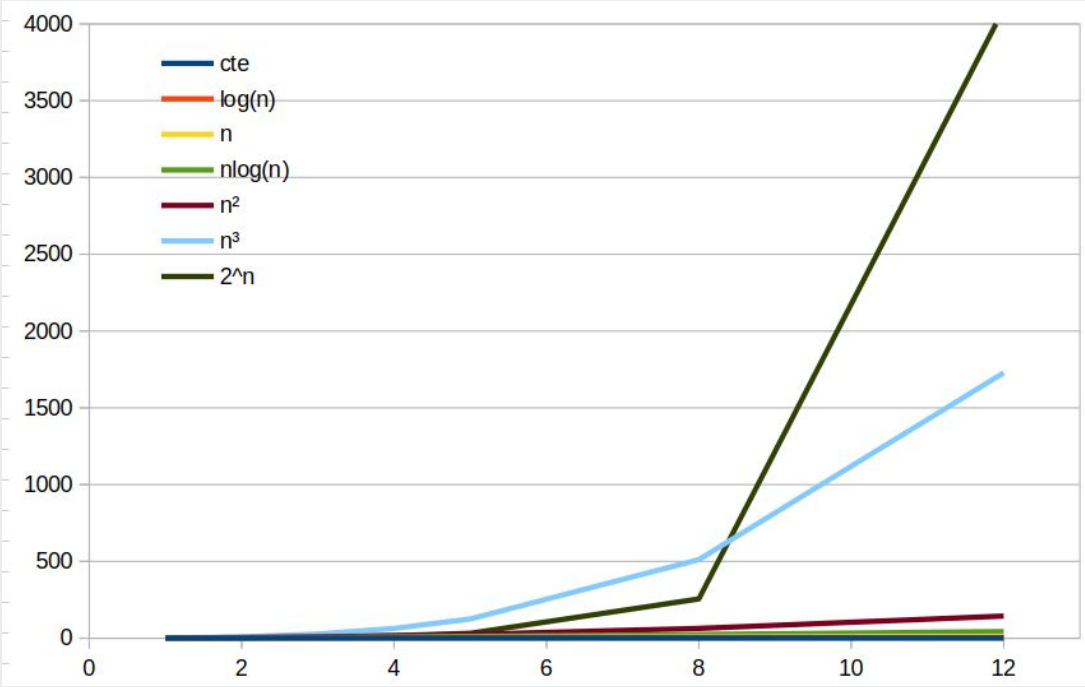
A Complexidade Assintótica fornece limites para a função de complexidade

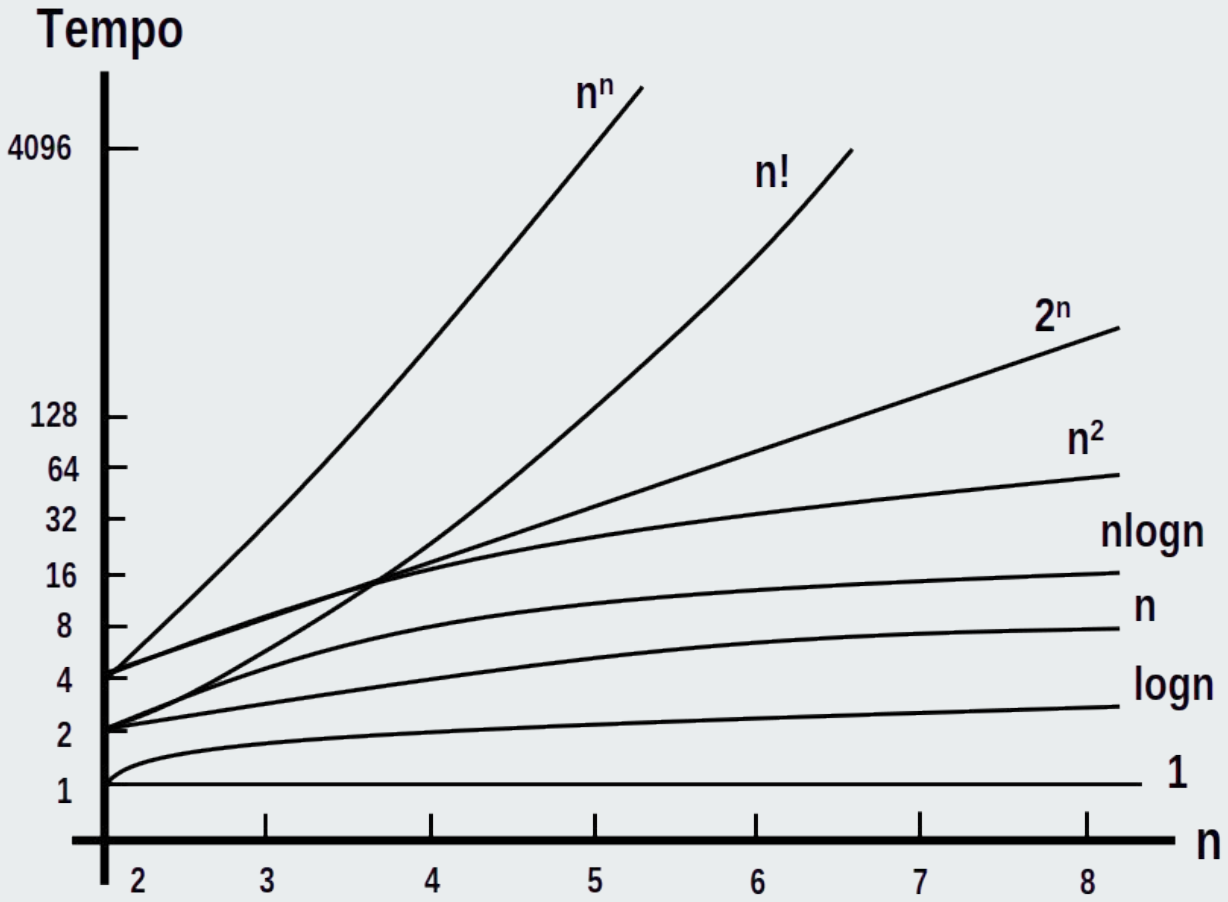
O - Big O
Θ - Theta
Ω - Ômega

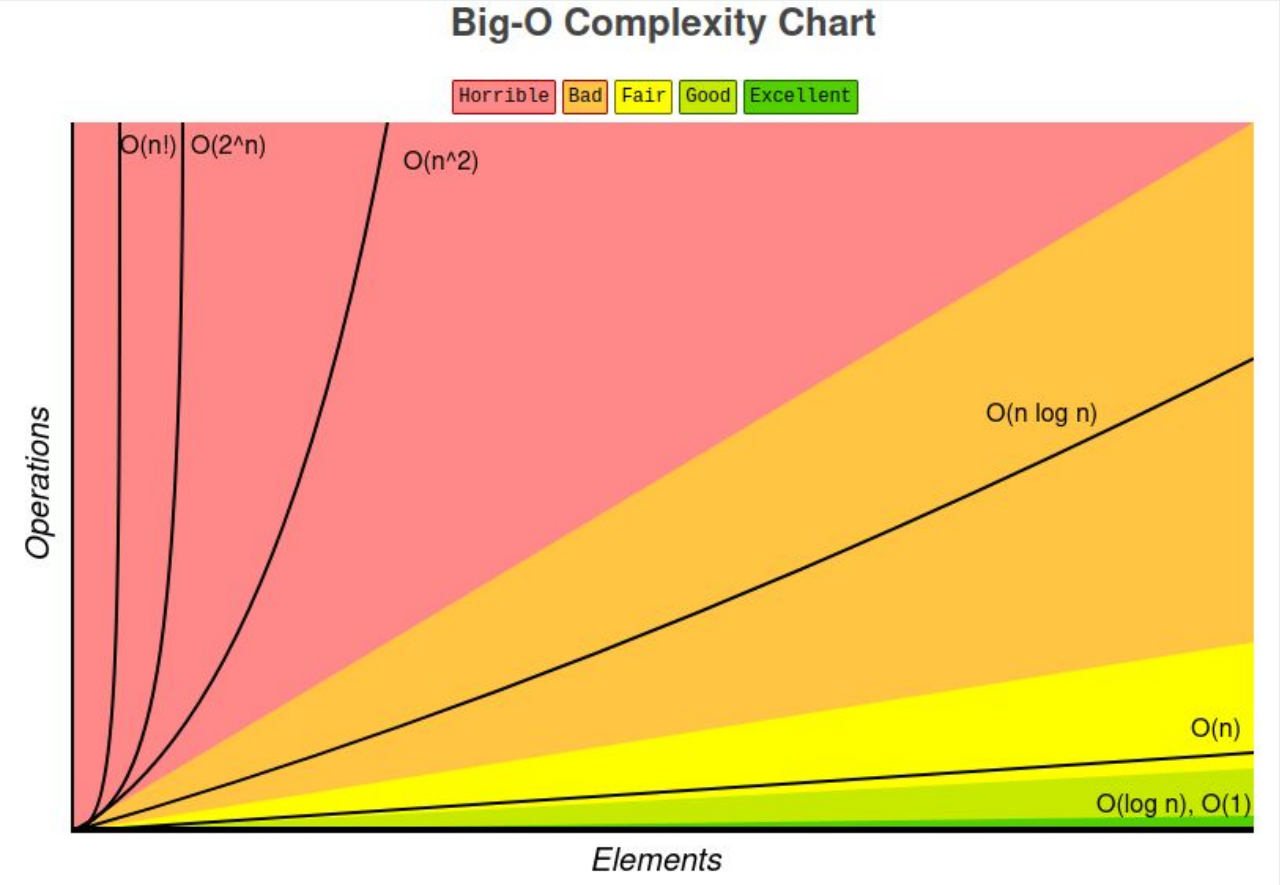
É uma abstração que permite analisar o comportamento de `T(n)`, quando `n` cresce indefinidamente



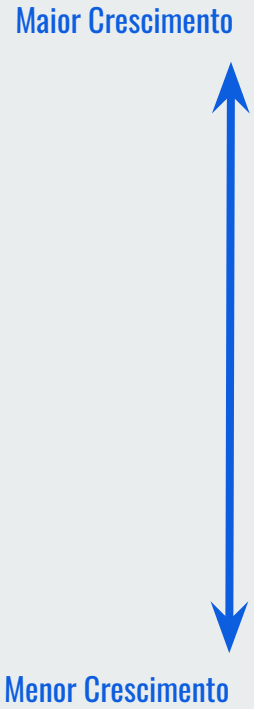
n	cte	log(n)	n	nlog(n)	n^2	n^3	2^n	n!
1	1,00	0,00	1,00	0,00	1,00	1,00	2,00	1,00
2	1,00	1,00	2,00	2,00	4,00	8,00	4,00	2,00
3	1,00	1,58	3,00	4,75	9,00	27,00	8,00	6,00
4	1,00	2,00	4,00	8,00	16,00	64,00	16,00	24,00
5	1,00	2,32	5,00	11,61	25,00	125,00	32,00	120,00
8	1,00	3,00	8,00	24,00	64,00	512,00	256,00	4,03E+04
12	1,00	3,58	12,00	43,02	144,00	1728,00	4,10E+03	4,79E+08




Tempo
4096
128
64
32
16
8
4
2
1
n^n
n!
2^n
n^2
nlogn
n
logn
1
n
2
3
4
5
6
7
8



https://www.bigocheatsheet.com/

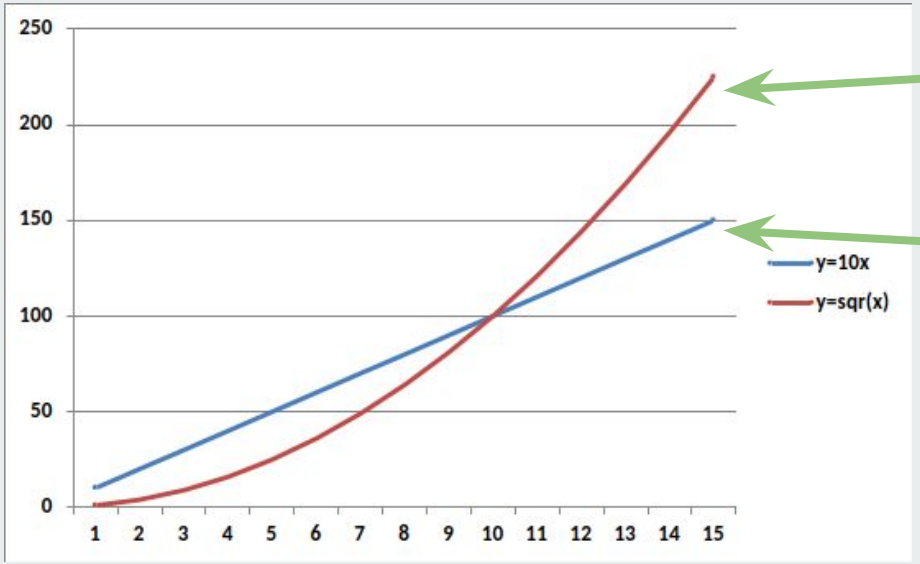


$O(n!)$	fatorial
$O(2^n)$	exponencial
$O(n^3)$	cúbica
$O(n^2)$	quadrática
$O(n \log n)$	log linear
$O(n)$	linear
$O(\log n)$	logarítmica
$O(1)$	constante

Notação Big O

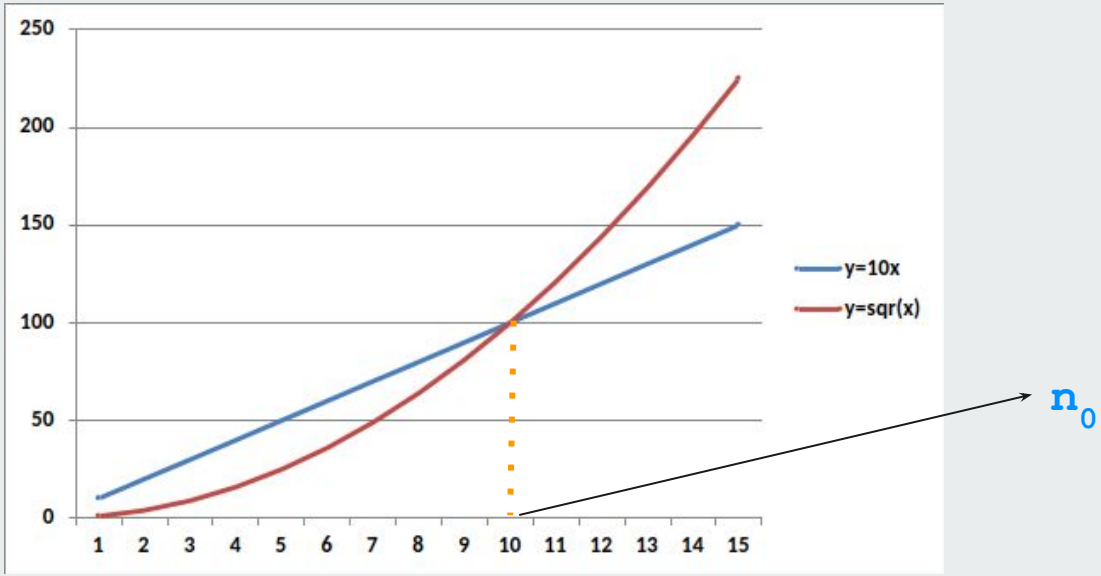
Considere duas funções, `f(n)` e `g(n)`, não negativas.

Diz-se que `f(n)` é de ordem `g(n)`, ou simplesmente `f(n)` é `O(g(n))` se o crescimento de `f(n)` é, no máximo, tão rápido quanto o crescimento de `g(n)`.



Notação Big O

Diz-se que f(n) é O(g(n)) se existe uma constante real positiva c e uma constante inteira positiva n_0 tais que $f(n) \leq cg(n)$, para todo valor de $n > n_0$.



f(n) é limitada superiormente por um múltiplo real positivo de g(n) para valores grandes de n.

Vamos analisar...

Diz-se que f(n) é O(g(n)) se existe uma constante real positiva c e uma constante inteira positiva n_0 tais que $f(n) \leq cg(n)$, para todo valor de $n > n_0$.

A função $f(n)=7tn + 2t$ é de ordem $g_1(n)=n$?

Vamos analisar...

Diz-se que f(n) é O(g(n)) se existe uma constante real positiva c e uma constante inteira positiva n_0 tais que $f(n) \leq cg(n)$, para todo valor de $n > n_0$.

A função $f(n)=7n + 2$ é de ordem $g_1(n)=n$?

outra forma...

Vamos analisar...

Diz-se que f(n) é O(g(n)) se existe uma constante real positiva c e uma constante inteira positiva n_0 tais que $f(n) \leq cg(n)$, para todo valor de $n > n_0$.

A função $f(n)=7n + 2$ é de ordem $g_2(n)=n^2$?

Vamos analisar...

Diz-se que f(n) é O(g(n)) se existe uma constante real positiva c e uma constante inteira positiva n_0 tais que $f(n) \leq cg(n)$, para todo valor de $n > n_0$.

A função $f(n) = 7n + 2$ é de ordem $g_3(n) = \sqrt{n}$?

Notação Big O

Diz-se que f(n) é O(g(n)) se existe uma constante real positiva `c` e uma constante inteira positiva n_0 tais que $f(n) \leq cg(n)$, para todo valor de $n > n_0$.

Esta definição equivale a dizer que: $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ **existe e é finito;**

1. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} \in \Re_+^*$ então $f(n) \in O(g(n))$

2. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ então $f(n) \in O(g(n))$

3. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = +\infty$ então $f(n) \notin O(g(n))$

A função $f(n)=7n+2$ é de ordem $g_1(n)=n$?

1. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} \in \Re_+^*$ então $f(n) \in O(g(n))$
2. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ então $f(n) \in O(g(n))$
3. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = +\infty$ então $f(n) \notin O(g(n))$

A função $f(n)=7n + 2$ é de ordem $g_2(n)=n^2$?

1. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} \in \Re_+^*$ então $f(n) \in O(g(n))$
2. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ então $f(n) \in O(g(n))$
3. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = +\infty$ então $f(n) \notin O(g(n))$

A função $f(n) = 7n + 2$ é de ordem $g_3(n) = \sqrt{n}$?

1. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} \in \Re_+^*$ então $f(n) \in O(g(n))$
2. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ então $f(n) \in O(g(n))$
3. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = +\infty$ então $f(n) \notin O(g(n))$

A função $f(n)=7n^2+3n-2$ é de ordem (a) $g_1(n)=n$ (b) $g_2(n)=n^2$ (c) $g_2(n)=n^3$?

1. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} \in \Re_+^*$ então $f(n) \in O(g(n))$
2. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ então $f(n) \in O(g(n))$
3. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = +\infty$ então $f(n) \notin O(g(n))$

Algumas Regras Básicas

- ➔ Termos de menor ordem não importam
- ➔ Constantes não importam
- ➔ Deve ser dada ênfase nas repetições de código
- ➔ Verificar a complexidade de funções e/ou métodos próprios da linguagem.

Regra da Soma

Se um algoritmo **A** se divide em duas partes independentes, **A1** e **A2**, com complexidades dadas por $T_1(n)$ e $T_2(n)$ de ordem $O(f(n))$ e $O(g(n))$, respectivamente, então $T(n) = T_1(n) + T_2(n)$ e **A** será de ordem $O(max\ \{f(n),\ g(n)\})$.

```
AlgoritmoA(n)
para i=1 até n
  imprima i

para i=1 até n
  para j=1 até n
    imprima i*j
```

Regra do Produto

Se um algoritmo **A** se divide em duas partes interdependentes, **A1** e **A2**, com complexidades dadas por $T_1(n)$ e $T_2(n)$ de ordem $O(f(n))$ e $O(g(n))$, respectivamente, então $T(n) = T_1(n) \times T_2(n)$ e **A** será de ordem $O(f(n) \times g(n))$.

```
AlgoritmoA(n)
para i=1 até n
   imprima (buscalista)
```

```
somaMatriz(A,B,n)
para i = 1 até n
   para j = 1 até n
      C[i,j] = A[i,j] + B[i,j]
   retorne(C)
```

Código em C++

```
int buscaLista2(vector<int> A, int x){
int tam = A.size();
sort(A.begin(),A.end());
int i = 0;
while (i<tam && A[i]< x){
    i = i+1;
}
if (i < tam && A[i]==x)
    return i;
return -1;
}
```

Código em C++

```
bool idadeRep(vector<int> Idade){
int tam = Idade.size();
int menor = 200;
for(int i = 0; i<tam; i++){
    if (Idade[i]<menor){
        menor = Idade[i];
    }
}
for(int i = 0; i<tam; i++){
    if (Idade[i]==menor){
        return true;
    }
}
return false;
```

Código em C++

```
bool idadeRep2(vector<int> Idade){
sort(Idade.begin(),Idade.end());
return Idade[0]==Idade[1];
}
```

E essa diferença no tempo de processamento é significativa?

Considere 5 algoritmos $A_1 ... A_5$ que resolvem um mesmo problema com um tempo de 1 ms em cada operação

$T_k(n)$ é a complexidade do algoritmo *K* na entrada *n*

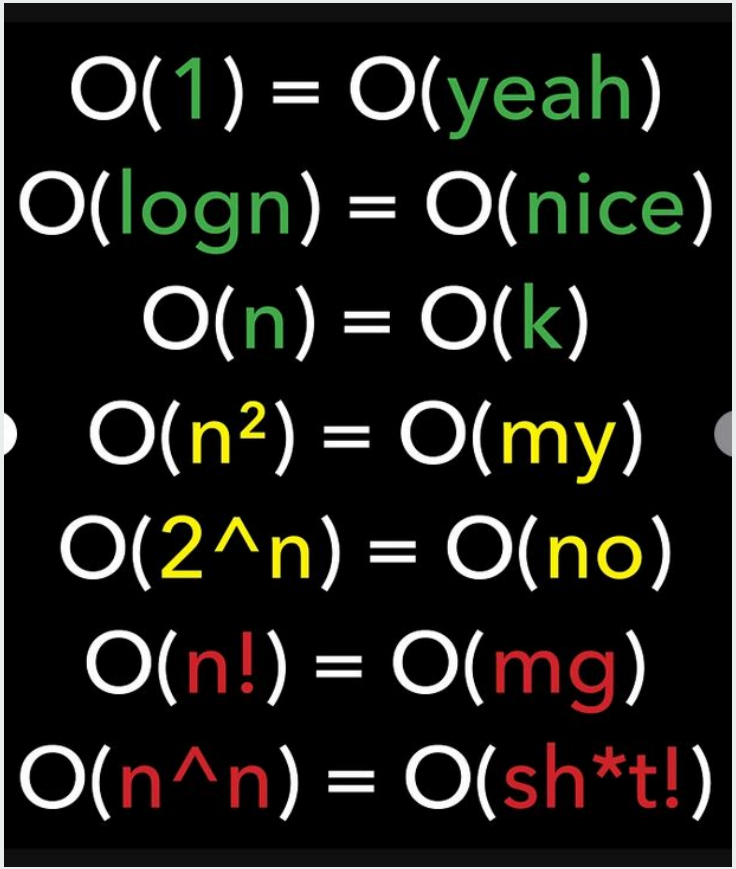
n	A_1 $T(n)=n$	A_2 $T(n)=n\log n$	A_3 $T(n)=n^2$	A_4 $T(n)=n^3$	A_5 $T(n)=2^n$
16	0,016s	0,064s	0,256s	4s	1m4s
32	0,032s	0,16s	1s	33s	46d
512	0,512s	9s	4m22s	1d13h	10^{137} Séculos

E essa diferença no tempo de processamento é significativa?

Considere 5 algoritmos $A_1 ... A_5$ que resolvem um mesmo problema com um tempo de 1 ps em cada operação

$T_k(n)$ é a complexidade do algoritmo K na entrada n

n	A_1 $T(n)=n$	A_2 $T(n)=n\log n$	A_3 $T(n)=n^2$	A_4 $T(n)=n^3$	A_5 $T(n)=2^n$
16	1,6E-11s	6,4E-11s	2,56E-10s	4E-09s	6,5E-08s
32	3,2E-11s	1,6E-10s	1 E-09s	3,3E-08s	4,3E-03s
512	5,12E-11s	4,6E-09s	2,6E-07s	1,3E-04s	10^{132} Séculos



https://www.bigocheatsheet.com/

Exercícios...

1. Implemente os códigos idadeRep e idadeRep2;
2. Crie instâncias aleatórias com idades (inteiros) variando entre 0 e 100, com tamanhos `n=100`, `n=1000`, `n=1000000`
3. Compute os tempos de processamento e compare



Dúvidas?!

